



**MATEMATICKO-FYZIKÁLNÍ
FAKULTA**
Univerzita Karlova

DIPLOMOVÁ PRÁCE

Martin Struk

Rozbíjení symetrií pro řešení splnitelnosti

Katedra algebry

Vedoucí diplomové práce: Mgr. Mikoláš Janota, Ph.D.

Studijní program: Matematika pro informační
technologie

Praha 2026

Prohlašuji, že jsem tuto diplomovou práci vypracoval(a) samostatně a výhradně s použitím citovaných pramenů, literatury a dalších odborných zdrojů. Beru na vědomí, že se na moji práci vztahují práva a povinnosti vyplývající ze zákona č. 121/2000 Sb., autorského zákona v platném znění, zejména skutečnost, že Univerzita Karlova má právo na uzavření licenční smlouvy o užití této práce jako školního díla podle §60 odst. 1 autorského zákona.

V dne

Podpis autora

Poděkování.

Název práce: Rozbíjení symetrií pro řešení splnitelnosti

Autor: Martin Struk

Katedra: Katedra algebry

Vedoucí diplomové práce: Mgr. Mikoláš Janota, Ph.D., Katedra algebry

Abstrakt: Abstrakt.

Klíčová slova: klíčové slovo, složitější fráze

Title: Symmetry Breaking Constraints for SAT Solving

Author: Martin Struk

Department: Department of algebra

Supervisor: Mgr. Mikoláš Janota, Ph.D., Department of algebra

Abstract: Abstract.

Keywords: key, words

Obsah

Úvod	6
1 Teoretická část	8
1.1 Základní definice	8
1.2 Kódování podmínek	10
1.3 Popis problémů a jejich kódování	11
1.3.1 Problém holubníku (Pigeonhole principle)	11
1.3.2 Problém N královen (N-queens problem)	12
1.3.3 Problém nalezení matice s řádky a sloupci se zadanými součty	13
1.3.4 Problém latinských čtverců	14
1.3.5 Problém společenských golfistů	15
2 Technická dokumentace	17
2.1 Generování vstupních instancí	17
2.2 Generování klauzulí pro rozbíjení symetrií	18
2.3 Pomocné funkce	19
2.4 Generování klauzulí z porovnání dvou vektorů proměnných	21
2.5 Generování klauzulí pro rozbíjení symetrií	23
3 Experimentální část	25
Závěr	26
Literatura	27
Seznam obrázků	28
Seznam tabulek	29
Seznam použitých zkratk	30
A Přílohy	31
A.1 První příloha	31

Úvod

Cílem této práce je implementovat nástroj, který pro vstupní SAT formuli vygeneruje klauzule pro rozbíjení symetrií pro zvolenou množinu permutací na proměnných, a dále zkoumat, jak různé výběry množin permutací ovlivňují celkový počet rozbitých symetrií, množství přidávaných klauzulí a celkově efektivitu nástroje.

Problém splnitelnosti booleovských formulí (SAT) je jedním ze základních rozhodovacích problémů. Cílem je určit, zda existuje ohodnocení proměnných zadané booleovské formule takové, že výsledný výraz je pravdivý. Přestože se nástroje pro řešení SAT (SAT solvery) instancí výrazně posunuly kupředu, složité instance často obsahují redundance a struktury, které stále příliš rozšiřují prohledávací prostor. Jedním ze zdrojů takové redundance jsou právě symetrie.

Symetrie v SAT formulích lze chápat jako permutace proměnných, které zachovávají pravdivost celé formule. Jako hlavní příklad symetrií v této práci budeme uvažovat záměnu ekvivalentních objektů ve vstupní úloze. Pro lepší představu si jako jednoduchý příklad uvedeme problém holubníku. Zde je cílem určit, zda je možné rozmístit zadaný počet holubů do zadaného počtu holubníků tak, že v každém holubníku bude nejvýše jeden holub. Na jednotlivé holuby ani holubníky nemáme žádné speciální požadavky, takže například pro 5 holubníků a 1 holuba máme 5 různých řešení, protože nezáleží na tom, který holubník vybereme. Přitom z jednoho řešení na druhé se můžeme dostat permutací označení jednotlivých holubníků, protože jednotlivé holubníky jsou ekvivalentní. V tomto smyslu jsou i samotná řešení ekvivalentní a SAT solveru by stačilo zkontrolovat jediné z nich, aby přišel na to, jestli jsou všechna pravdivá, nebo nepravdivá. Avšak právě podobné permutované varianty stejného řešení SAT solvery často zbytečně procházejí mnohokrát během jednoho řešení, čímž se dramaticky zvětšuje prohledávací prostor. V ideálním případě SAT solver z každé třídy ekvivalencí řešení zkontroluje vždy nejvýše jedno řešení. K tomuto stavu se chceme přidáváním klauzulí co nejvíce přiblížit a říkáme mu úplné rozbití symetrie.

Úplné rozbití všech symetrií je ale v realitě většinou příliš náročné. SAT solver sice v takovém případě poběží kratší dobu, ale samotné generování všech potřebných klauzulí pro rozbíjení symetrií zabere příliš mnoho času. Musíme tedy generovat určitou podmnožinu klauzulí pro rozbíjení symetrií, která ale stále rozbije dostatečné množství symetrií. Tomuto přístupu říkáme částečné rozbití symetrie.

Jedním z nejčastěji používaných způsobů rozbíjení symetrií je metoda Lex-Leader. Zde pro každou třídu ekvivalencí řešení definujeme jedno jako kanonické řešení. Toho docílíme přidáním omezujících podmínek, které splňuje pouze kanonické řešení. Základem je nejdříve si zvolit uspořádání na proměnných. Z tohoto uspořádání odvodíme uspořádání na celých ohodnoceních stejně jako u lexikografického uspořádání. Máme tedy

$$x_1 \prec x_2 \prec \dots \prec x_n$$

a pro každé ohodnocení x a každou permutaci π přidáme omezující podmínku

$$x \leq_{\text{lex}} \pi(x).$$

Vybrané kanonické řešení je vždy lexikograficky nejmenší a tím pádem jako jediné z dané třídy ekvivalencí bude splňovat všechny přidané podmínky. Tímto zřejmě dosáhneme úplného rozbití symetrií. Jak již ale bylo řečeno, v tomto případě bychom potřebovali přidat příliš mnoho omezujících podmínek. Z tohoto důvodu se budeme zaměřovat na částečné rozbití symetrií, kdy se omezující podmínky generují pouze pro vybranou podmnožinu permutací.

1 Teoretická část

Background, permutace, generátory, satsuma, breakid, detekují symetrie ale není jasné proč fungují dobře generátory

Definice: budeme používat 2D binární matice, seřazení do vektoru a proměnné v řádcích

Definice patternu, odkaz na paper, popis problémů

1.1 Základní definice

Jelikož se budeme zabývat symetriemi, hlavní objekty, na které se budeme soustředit, budou samozřejmě podgrupy symetrické grupy. Prvky, které budeme permutovat, budou proměnné ze zadaných SAT formulí. Ty můžeme reprezentovat jednoduše jako čísla $\{1, \dots, n\}$, takže symetrickou grupu na těchto prvcích můžeme značit S_n .

Budeme předpokládat, že symetrie zadaného problému známe již ze zadání. Je to z toho důvodu, že budeme chtít navázat na jednu konkrétní informaci z nástrojů jako je *BREAKID* nebo *SATSUMA* (odkaz na papery). Tyto nástroje dostanou na vstupu SAT formulí a snaží se v ní detekovat určité typy symetrií. Pokud symetrii detekují, tím pádem najdou určitou podgrupu $G \subseteq S_n$, rozbíjí symetrie generováním Lex-Leader klauzulí $x \leq_{\text{lex}} \pi(x)$ pro π z množiny generátorů G , které získají při detekci symetrie. Zmíněnou informací je, že takto získané klauzule dávají velmi dobré částečné rozbití symetrie, ale ani samotným autorům těchto nástrojů není jasné, proč tomu tak je. V této práci chceme tedy zkoumat, jestli vlastnost, že určitá množina permutací generuje celou podgrupu odpovídající symetrii daného problému, znamená, že s ní dosáhneme lepšího částečného rozbití symetrie. Pro úplnost si na jednoduchém příkladu ukážeme, že množinou generující danou grupu permutací nemusíme dosáhnout úplného rozbití symetrie, což by někoho na první pohled mohlo u generátorů napadnout.

Příklad. Použijeme příklad již zmíněný v úvodu. Uvažujme tedy problém holubníku s pěti holubníky a jedním holubem. Máme tedy celkem pět proměnných, které budeme nyní pro přehlednost značit x_1, x_2, x_3, x_4, x_5 . Hodnoty těchto proměnných mohou být pouze 1 nebo 0 a hodnota 1 znamená, že holub je v holubníku odpovídajícímu danému indexu. Z našich pěti proměnných tedy vždy musí právě jedna nabývat hodnoty 1 a právě čtyři musí nabývat hodnoty 0. Holubníky jsou si ekvivalentní a tím pádem můžeme proměnné libovolně permutovat. Pracujeme tedy s grupou S_5 (kde permutace indexů odpovídá permutaci proměnných s těmito indexy). Pro tu vezmeme množinu generátorů $\{(1,2), (1,5,4,3,2)\}$. Označme si je $\pi_1 := (1,2)$ a $\pi_2 := (1,5,4,3,2)$. Nyní dostaneme dvě podmínky:

$$\begin{aligned} x &= (x_1, x_2, x_3, x_4, x_5) \leq_{\text{lex}} (x_2, x_1, x_3, x_4, x_5) = \pi_1(x) \\ x &= (x_1, x_2, x_3, x_4, x_5) \leq_{\text{lex}} (x_2, x_3, x_4, x_5, x_1) = \pi_2(x) \end{aligned}$$

Dosazením všech možných ohodnocení dostaneme (první sloupec představuje

dosazení pro π_1 a druhý sloupec pro π_2):

$$\begin{array}{ll} (1,0,0,0,0) \leq_{\text{lex}} (0,1,0,0,0) & (1,0,0,0,0) \leq_{\text{lex}} (0,0,0,0,1) \\ (0,1,0,0,0) \leq_{\text{lex}} (1,0,0,0,0) & (0,1,0,0,0) \leq_{\text{lex}} (1,0,0,0,0) \\ (0,0,1,0,0) \leq_{\text{lex}} (0,0,1,0,0) & (0,0,1,0,0) \leq_{\text{lex}} (0,1,0,0,0) \\ (0,0,0,1,0) \leq_{\text{lex}} (0,0,0,1,0) & (0,0,0,1,0) \leq_{\text{lex}} (0,0,1,0,0) \\ (0,0,0,0,1) \leq_{\text{lex}} (0,0,0,0,1) & (0,0,0,0,1) \leq_{\text{lex}} (0,0,0,1,0) \end{array}$$

Pro nás bude vždy platit, že $0 \leq_{\text{lex}} 1$ a že proměnné dříve v pořadí mají větší váhu než proměnné dále. Kanonickým řešením pro nás tedy zde je $(0,0,0,0,1)$. Pokud se nyní podíváme na nerovnosti výše, vidíme, že v obou sloupcích je neplatný první řádek. To znamená, že jsme přidáním podmínek z permutací π_1, π_2 odstranili jedno ekvivalentní řešení z pěti, a to $(1,0,0,0,0)$ (k tomu by stačilo, aby byl neplatný řádek jen v jednom ze sloupců). Pro úplné rozbití symetrie ale potřebujeme odstranit čtyři z pěti těchto ohodnocení.

Analogickým postupem můžeme ověřit, že ale například pro množinu generátorů $\{(1,2), (1,2,3,4,5)\}$ úplné rozbití symetrie dostaneme a zbyde nám jako jediné možné řešení $(0,0,0,0,1)$.

V této práci se zaměříme především na problémy, jejichž řešení lze reprezentovat binární maticí. Veškeré postupy je možné přímočaře zobecnit i do vícerozměrných problémů, jen se zpravidla dramaticky zvedne počet proměnných, symetrií a celkově náročnost vyřešení problému.

Definice 1. *Nechť A je $m \times n$ matice. Pak $\text{vec}(A)$ bude značit vektor délky $m \cdot n$, který získáme konkatencí všech řádků matice za sebe.*

Definice 2. *Nechť A, B jsou $m \times n$ binární matice a necht $a := \text{vec}(A), b := \text{vec}(B)$. Pak budeme uvažovat následující uspořádání $\leq$ definované jako:*

$$A \leq B \text{ právě tehdy, když } a \leq_{\text{lex}} b$$

Spíše než abychom ale pracovali s již známými maticemi řešení, budeme chtít lexikograficky menší řešení hledat. Místo binárních matic budou tedy matice obsahovat proměnné a pro každé ohodnocení těchto proměnných dostaneme konkrétní binární matici. Uspořádání $\leq$ z předchozí definice nám poté bude dávat podmínky, které musí ohodnocení proměnných splňovat.

Příklad. Vraťme se opět k problému holubníku. Nyní uvažujme, že máme 3 holuby a 3 holubníky. Vytvoříme si matici H , ve které budou řádky odpovídat jednotlivým holubům a sloupce holubníkům. Prvky $h_{i,j}$ matice nám pak říkají, jestli i -tý holub je v j -tém holubníku. Základní podmínky, které musíme zakódovat do SAT klauzulí, jsou, že každý holub musí být alespoň v jednom holubníku a v každém holubníku je nejvýše jeden holub. Toto kódování si popíšeme později. Nyní je podstatné, že máme následující matici:

$$H = \begin{pmatrix} x_1 & x_2 & x_3 \\ x_4 & x_5 & x_6 \\ x_7 & x_8 & x_9 \end{pmatrix}$$

H musí z výše uvedených podmínek mít v každém řádku alespoň jednu 1 a v každém sloupci nejvýše jednu 1. Již víme, že holubníky můžeme libovolně permutovat. Stejně tak zřejmě můžeme permutovat i holuby. V tomto případě tedy pracujeme s grupou $S_3 \times S_3 \subseteq S_9$. Vezmeme-li například pro řádky (holuby) permutaci $\pi_1 = (1,2,3)$ a pro sloupce (holubníky) permutaci $\pi_2 = (2,1,3)$, dostaneme pro $\pi := (\pi_1, \pi_2)$

$$\pi(H) = \begin{pmatrix} x_8 & x_9 & x_7 \\ x_2 & x_3 & x_1 \\ x_5 & x_6 & x_4 \end{pmatrix}$$

a přidáme podmínku $H \leq \pi(H)$. Což znamená

$$\begin{aligned} \text{vec}(H) &\leq_{\text{lex}} \text{vec}(\pi(H)) \text{ ,neboli} \\ (x_1, x_2, x_3, x_4, x_5, x_6, x_7, x_8, x_9) &\leq_{\text{lex}} (x_8, x_9, x_7, x_2, x_3, x_1, x_5, x_6, x_4). \end{aligned}$$

Dále už bychom mohli postupovat analogicky prvnímu příkladu s jedním holubem a pěti holubníky.

1.2 Kódování podmínek

V této části se podíváme blíže na to, jak budeme kódovat podmínky $\leq_{\text{lex}}$ jako SAT klauzule. Využijeme k tomu takzvané *vzory* z (odkaz na paper) (anglicky *patterns*). Idea je taková, že pro podmínku $x \leq_{\text{lex}} \pi(x)$ budeme naopak postupně sestavovat podmínku $\pi(x) <_{\text{lex}} x$, kterou poté znegujeme. Postupným sestavováním se myslí, že si v každém kroku zafixujeme pozici ve vektorech, na které se mají poprvé lišit (respektive první pozice, na které je proměnná v $\pi(x)$ ostře menší než proměnná v x). To nám zároveň říká, že na všech předchozích pozicích se vektory musí rovnat. Z těchto vlastností pak odvodíme hledaný vzor.

Definice 3. *Nechť $a = (a_1, \dots, a_n)$ a $b = (b_1, \dots, b_n)$. Pak řekneme, že a je menší než b na i -té pozici, pokud*

$$(a_1 = b_1) \wedge \dots \wedge (a_{i-1} = b_{i-1}) \wedge (a_i < b_i).$$

Tuto vlastnost budeme značit $a <_{\text{lex}}^i b$.

Poznámka. Jelikož pracujeme s binárními proměnnými, $(a_i < b_i)$ znamená jednoduše $(a_i = 0) \wedge (b_i = 1)$.

Poznámka.

$$a <_{\text{lex}}^i b \Leftrightarrow \bigvee_{i=1}^n a <_{\text{lex}}^i b$$

Definice 4. *Nechť π je permutace a $a = (a_1, \dots, a_n)$. Pak vzorem $\text{pat}_i(\pi)$ rozumíme aplikování rovností a nerovností z $\pi(a) <_{\text{lex}}^i a$ na vektor a .*

Příklad. Uvažujme vektory z předchozího příkladu a mějme $\text{vec}(\pi(H)) <_{\text{lex}}^i \text{vec}(H)$ pro $i = 4$, neboli

$$(x_8, x_9, x_7, x_2, x_3, x_1, x_5, x_6, x_4) <_{\text{lex}}^4 (x_1, x_2, x_3, x_4, x_5, x_6, x_7, x_8, x_9).$$

Tím tedy dostáváme podmínky $(x_8 = x_1), (x_9 = x_2), (x_7 = x_3)$ a $(x_2 < x_4)$. To znamená, že $x_2 = 0, x_4 = 1$ a zároveň také z druhé rovnosti $x_9 = 0$. Pro odlišení dále označíme $A := x_1 (= x_8)$ a $B := x_3 (= x_7)$. Dosazením (aplikováním) těchto hodnot do $vec(H)$ dostaneme

$$pat_4(\pi) = (A, 0, B, 1, x_5, x_6, B, A, 0).$$

Nyní pokud libovolný vektor v odpovídá tomuto vzoru, tak o něm automaticky víme, že $v <_{\text{lex}} vec(H)$. Jelikož A, B, x_5, x_6 mohou být libovolná, pokryli jsme tímto vzorem celkem $2^4 = 16$ různých vektorů.

1.3 Popis problémů a jejich kódování

1.3.1 Problém holubníku (Pigeonhole principle)

Jako první se podíváme na již několikrát zmiňovaný problém holubníku. Princip holubníku nám říká, že pokud máme m předmětů (holubů), chceme je rozmístit do n kontejnerů (holubníků) a navíc $m > n$, pak musí alespoň jeden kontejner obsahovat více než jeden předmět. Přestože se toto tvrzení zdá být zřejmé, pomáhá vysvětlit spousta zdaleka ne tak zřejmých tvrzení. Tento princip přeformulujeme do tvaru problému následovně. Pro zadaný počet holubů m a počet holubníků n musíme holuby rozmístit do holubníků tak, aby každý holub byl alespoň v jednom holubníku a aby v každém holubníku byl nejvýše jeden holub. Většinou se uvažuje $m = n + 1$, takže je problém nesplnitelný. Přestože se výsledek opět zdá být zřejmý, pro SAT solvery je řešení tohoto problému bez rozbíjení symetrií velmi náročné. Proto se tento problém velmi často využívá jako základní benchmark právě pro testování rozbíjení symetrií.

Kódování tohoto problému do SAT formule je přímočaré. Stejně jako v jednom z předchozích příkladů budeme mít matici binárních proměnných, kde řádky budou odpovídat holubům a sloupce holubníkům. Máme tedy $m \times n$ matici. Proměnné jsou x_{in+j+1} , kde i je řádek (holub) a j je sloupec (holubník) a $x_{in+j+1} = 1$, pokud i -tý holub je umístěn do j -tého holubníku, a $x_{in+j+1} = 0$ jinak. Řádky a sloupce budeme v této sekci vždy indexovat od 0. Kódování podmínek pak vypadá následovně:

Poznámka. (Problém holubníku jako SAT formule) Každý holub v alespoň jednom holubníku:

$$\bigwedge_{i=0}^{m-1} (x_{in+1} \vee x_{in+2} \vee \cdots \vee x_{in+n})$$

V každém holubníku nejvýše jeden holub:

$$\bigwedge_{j=0}^{n-1} \bigwedge_{0 \leq i < k \leq m-1} (\neg x_{in+j+1} \vee \neg x_{kn+j+1})$$

Jak již bylo také zmíněno, symetrie zde leží v tom, že můžeme mezi sebou permutovat zvlášť holuby a zvlášť holubníky. Můžeme tedy permutovat buď řádky, sloupce, nebo obojí nezávisle na sobě.

1.3.2 Problém N královen (N-queens problem)

Dalším z vybraných problémů je známý problém N královen. Zde dostaneme na vstupu jediný parametr n a za úkol máme rozmístit n šachových figurek královen na $n \times n$ šachovnici tak, aby se žádné dvě z nich vzájemně neohrožovaly. Řešení tohoto problému pro SAT solvery není až tak náročné i bez rozbíjení symetrií a jedno řešení zvládají rychle najít i pro relativně velká n . Avšak celkový počet řešení se zvyšujícím se n stoupá dramaticky a my tím pádem můžeme dobře sledovat, o kolik se celkový počet řešení po rozbíjení symetrií sníží.

Kódování problému N královen do SAT formule také není složité. Vytvoříme $n \times n$ matici binárních proměnných, kde jednotlivé pozice odpovídají pozicím na šachovnici. Pro i -tý řádek a j -tý sloupec pak $x_{(i-1)n+j} = 1$, pokud jsme umístili královnu na pozici (i, j) , a $x_{(i-1)n+j} = 0$ jinak. Nejjednodušší kódování pak dostaneme pro podmínky právě jedné královny v každém řádku, sloupci a na každé diagonále následovně:

Poznámka. (Problém N královen jako SAT formule) V každém řádku alespoň jedna královna:

$$\bigwedge_{r=0}^{n-1} \left(\bigvee_{c=0}^{n-1} x_{rn+c+1} \right)$$

V každém řádku nejvýše jedna královna:

$$\bigwedge_{r=0}^{n-1} \bigwedge_{0 \leq c_1 < c_2 \leq n-1} (\neg x_{rn+c_1+1} \vee \neg x_{rn+c_2+1})$$

V každém sloupci alespoň jedna královna:

$$\bigwedge_{c=0}^{n-1} \left(\bigvee_{r=0}^{n-1} x_{rn+c+1} \right)$$

V každém sloupci nejvýše jedna královna:

$$\bigwedge_{c=0}^{n-1} \bigwedge_{0 \leq r_1 < r_2 \leq n-1} (\neg x_{r_1n+c+1} \vee \neg x_{r_2n+c+1})$$

Nejvýše jedna královna diagonálně směrem doprava nahoru:

$$\bigwedge_{d=-(n-1)}^{n-1} \bigwedge_{\substack{0 \leq r_1 < r_2 \leq n-1 \\ 0 \leq r_1-d < n, 0 \leq r_2-d < n}} (\neg x_{r_1n+(r_1-d)+1} \vee \neg x_{r_2n+(r_2-d)+1})$$

Nejvýše jedna královna diagonálně směrem doprava dolů:

$$\bigwedge_{s=0}^{2n-2} \bigwedge_{\substack{0 \leq r_1 < r_2 \leq n-1 \\ 0 \leq s-r_1 < n, 0 \leq s-r_2 < n}} (\neg x_{r_1n+(s-r_1)+1} \vee \neg x_{r_2n+(s-r_2)+1})$$

Kvůli podmínkám na diagonály u tohoto problému zřejmě nemůžeme libovolně permutovat řádky a sloupce. Avšak problém symetrie samozřejmě obsahuje. Z jednoho řešení můžeme získat několik dalších ekvivalentních řešení pomocí rotací a reflexí celé šachovnice. Zastoupení dalšího typu symetrií je dalším z důvodů zařazení problému N královen.

1.3.3 Problém nalezení matice s řádky a sloupci se zadanými součty

Pro následující problém může už být zadání zajímavější. Na vstupu dostaneme množinu čísel K a množinu čísel L , kde $m := |K|$, $n := |L|$, prvky K budeme značit k_r pro $r \in \{1, \dots, m\}$ a prvky L budeme značit l_c pro $c \in \{1, \dots, n\}$. Každý prvek K odpovídá zadání pro jeden řádek a každý prvek L odpovídá zadání pro jeden sloupec. Máme tedy $m \times n$ matici binárních proměnných. Naším úkolem pak je přiřadit proměnným hodnoty 0 a 1 tak, aby součet hodnot v r -tém řádku byl právě k_r , a aby součet hodnot v c -tém sloupci byl právě l_c .

Kódování je v tomto případě už také složitější. Podmínky na právě k_r jedniček v řádku a právě l_c jedniček ve sloupci se sice ve své podstatě podobají podmínkám na právě jednu jedničku v řádku/sloupci z předchozích problémů, ale počet klauzulí je zpravidla výrazně vyšší, protože přidáváme klauzuli pro každou kombinaci určitého počtu proměnných. Existují efektivnější kódování, ale těmi se v této práci nebudeme zabývat, jelikož problémy zde slouží především k testování. Hlavní ideou práce je, že na vstupu dostaneme SAT formuli a typ symetrie, se kterými pracujeme dále. Tím pádem nemáme zaručeno, že kódování je jakkoliv optimalizované, a zároveň ho sami optimalizovat nemůžeme, jelikož nevíme, co za problém SAT formule představuje. Námi použité kódování pak bude vypadat následovně:

Poznámka. (Problém nalezení matice s řádky a sloupci se zadanými součty jako SAT formule) Alespoň k_r jedniček v řádku r :

$$\bigwedge_{r=0}^{m-1} \bigwedge_{\substack{S \subseteq \{0, \dots, n-1\} \\ |S|=n-k_r+1}} \left(\bigvee_{c \in S} x_{rn+c+1} \right)$$

Nejvýše k_r jedniček v řádku r :

$$\bigwedge_{r=0}^{m-1} \bigwedge_{\substack{S \subseteq \{0, \dots, n-1\} \\ |S|=k_r+1}} \left(\bigvee_{c \in S} \neg x_{rn+c+1} \right)$$

Alespoň l_c jedniček ve sloupci c :

$$\bigwedge_{c=0}^{n-1} \bigwedge_{\substack{T \subseteq \{0, \dots, m-1\} \\ |T|=m-l_c+1}} \left(\bigvee_{r \in T} x_{rn+c+1} \right)$$

Nejvýše l_c jedniček ve sloupci c :

$$\bigwedge_{c=0}^{n-1} \bigwedge_{\substack{T \subseteq \{0, \dots, m-1\} \\ |T|=l_c+1}} \left(\bigvee_{r \in T} \neg x_{rn+c+1} \right)$$

Symetrie pro tento problém jsou také řádkové a sloupcové jako u problému holubníku, avšak permutovat mezi sebou můžeme jen řádky se stejnými požadovanými součty a stejně tak jen sloupce se stejnými požadovanými součty.

1.3.4 Problém latinských čtverců

Další z velmi známých problémů je problém latinských čtverců. Cílem je pro vstupní parametr n rozmístit do $n \times n$ matice hodnoty $1, \dots, n$ tak, aby se v každém řádku a v každém sloupci vyskytovala každá hodnota právě jednou. Jelikož proměnné mohou nabývat pouze 0, nebo 1, je tento problém na první pohled trojrozměrný. Nejjednodušší reprezentací se nabízí použití proměnných $x_{r,c,v}$, kde každá tato proměnná říká, jestli na pozici v r -tém řádku a v c -tém sloupci se vyskytuje hodnota v . My bychom ale chtěli opět pracovat s dvourozměrnou maticí. Proto si pro každou hodnotu v vrstvy trojrozměrné matice rozdělíme na n binárních matic $n \times n$ a ty poskládáme pod sebe do jedné dvourozměrné matice. Výsledkem tedy bude $n^2 \times n$ binární matice, kde proměnná $x_{vn^2+rn+c+1}$ odpovídá proměnné $x_{r,c,v}$ výše. Pro přehlednost však v rámci této sekce zůstaneme u zápisu pomocí proměnných ve tvaru $x_{r,c,v}$.

Kódování pak dostaneme již dobře známým postupem, kdy si podmínky pro právě jednu hodnotu na každé pozici, každou hodnotu právě jednou v každém řádku a každou hodnotu právě jednou v každém sloupci rozdělíme na část „alespoň jednou” a na část „nejvýše jednou”. Ty zapíšeme analogicky zápisu z předchozích problémů. Kódování pak vypadá následovně:

Poznámka. Problém latinských čtverců jako SAT formule Alespoň jedna hodnota na každé pozici:

$$\bigwedge_{r=0}^{n-1} \bigwedge_{c=0}^{n-1} \left(\bigvee_{v=0}^{n-1} x_{r,c,v} \right)$$

Nejvýše jedna hodnota na každé pozici:

$$\bigwedge_{r=0}^{n-1} \bigwedge_{c=0}^{n-1} \bigwedge_{0 \leq v_1 < v_2 < n} (\neg x_{r,c,v_1} \vee \neg x_{r,c,v_2})$$

Každá hodnota alespoň jednou v každém řádku:

$$\bigwedge_{r=0}^{n-1} \bigwedge_{v=0}^{n-1} \left(\bigvee_{c=0}^{n-1} x_{r,c,v} \right)$$

Každá hodnota nejvýše jednou v každém řádku:

$$\bigwedge_{r=0}^{n-1} \bigwedge_{v=0}^{n-1} \bigwedge_{0 \leq c_1 < c_2 < n} (\neg x_{r,c_1,v} \vee \neg x_{r,c_2,v})$$

Každá hodnota alespoň jednou v každém sloupci:

$$\bigwedge_{c=0}^{n-1} \bigwedge_{v=0}^{n-1} \left(\bigvee_{r=0}^{n-1} x_{r,c,v} \right)$$

Každá hodnota nejvýše jednou v každém sloupci:

$$\bigwedge_{c=0}^{n-1} \bigwedge_{v=0}^{n-1} \bigwedge_{0 \leq r_1 < r_2 < n} (\neg x_{r_1,c,v} \vee \neg x_{r_2,c,v})$$

U latinských čtverců můžeme zřejmě nezávisle na sobě permutovat řádky, sloupce a i hodnoty, které na pozice zapisujeme. U tohoto problému je dobře známé, že i pro malá n je celkový počet řešení obrovský. Například pro $n = 5$ existuje 161280 řešení, ale pro $n = 6$ jich existuje již 812851200. Tyto počty jsou však počítány bez jakéhokoliv rozbíjení symetrií a díky symetriím v řádcích, sloupcích i v hodnotách můžeme očekávat, že jich je tím pádem velmi velké množství ekvivalentních. V naší reprezentaci $n^2 \times n$ binární maticí musíme řádky permutovat vždy zvlášť v každé z n disjunktních $n \times n$ binárních podmatic (celá podmatice musí vždy odpovídat pouze jedné z hodnot a pokud permutujeme řádky u jedné z podmatic, musíme je stejně permutovat i u všech podmatic odpovídajícím ostatním hodnotám) a symetrie na hodnotách rozbíjíme permutacemi celých $n \times n$ binárních podmatic představujících jednotlivé hodnoty. Sloupce můžeme permutovat běžně bez žádných speciálních omezení.

1.3.5 Problém společenských golfistů

Posledním problémem, kterým se budeme zabývat je problém sociálních golfistů. Zde dostaneme na vstupu počet týdnů W , počet skupin G a počet hráčů v každé skupině ppg . Celkový počet hráčů si označíme P a musí tedy platit $P = ppg * G$. Za úkol pak máme rozvrhnout hry golfu na W týdnů tak, aby každý týden hrál každý hráč v některé ze skupin, a zároveň aby žádná dvojice hráčů nebyla společně ve stejné skupině za všechny týdny více než jednou. Tento problém ze zdá být ze všech uvedených problémů nejzajímavější a zřejmě je opět trojrozměrný. My však opět použijeme zápis dvourozměrnou binární maticí z problému latinských čtverců. Uvažujeme $P \times G$ binární podmatice, kterých pod sebe poskládáme W . Výslednou maticí je $(P \cdot W) \times G$ binární matice, ve které proměnné $x_{wPG+pg+g+1}$ říkají, jestli hráč p hraje ve skupině g ve w -tý týden. Pro přehlednost v této sekci zapisujeme tyto proměnné podobně jako v předchozím problému ve tvaru $x_{w,p,g}$.

Kódování se převážně opět podobá kódování předchozích problémů. Zásadně se odlišuje pouze podmínkou na to, že se žádný pár hráčů nemůže potkat dvakrát. Ta je zde opět podobně jako u problému nalezení matice s řádky a sloupci se zadanými součty zakódována nejjednodušším způsobem, přestože existují efektivnější kódování. Výsledné kódování je pak následující:

Poznámka. Problém společenských golfistů jako SAT formule Každý hráč v alespoň jedné skupině každý týden:

$$\bigwedge_{w=0}^{W-1} \bigwedge_{p=0}^{P-1} \left(\bigvee_{g=0}^{G-1} x_{w,p,g} \right)$$

Každý hráč v nejvýše jedné skupině každý týden:

$$\bigwedge_{w=0}^{W-1} \bigwedge_{p=0}^{P-1} \bigwedge_{0 \leq g_1 < g_2 < G} (\neg x_{w,p,g_1} \vee \neg x_{w,p,g_2})$$

Každá skupina má alespoň ppg hráčů každý týden:

$$\bigwedge_{w=0}^{W-1} \bigwedge_{g=0}^{G-1} \bigwedge_{\substack{S \subseteq P \\ |S|=P-ppg+1}} \left(\bigvee_{p \in S} x_{w,p,g} \right)$$

Každá skupina má nejvýše ppg hráčů každý týden:

$$\bigwedge_{w=0}^{W-1} \bigwedge_{g=0}^{G-1} \bigwedge_{\substack{S \subseteq P \\ |S|=ppg+1}} \left(\bigvee_{p \in S} \neg x_{w,p,g} \right)$$

Žádný pár hráčů se nepotká dvakrát:

$$\bigwedge_{0 \leq i < j < P} \bigwedge_{0 \leq w_1 < w_2 < W} \bigwedge_{g_1=0}^{G-1} \bigwedge_{g_2=0}^{G-1} (\neg x_{w_1,i,g_1} \vee \neg x_{w_1,j,g_1} \vee \neg x_{w_2,i,g_2} \vee \neg x_{w_2,j,g_2})$$

Ekvivalentní jsou si zde vzájemně hráči, skupiny i týdny. Všechny parametry tedy opět můžeme nezávisle na sobě libovolně permutovat a získáme ekvivalentní řešení. Permutace v naší dvourozměrné reprezentaci opět musíme provádět analogicky případu u problému latinských čtverců.

2 Technická dokumentace

generování kódování problémů, parametry, ukládání do souboru, formát v souboru

hlavní kód, množiny permutací, generování klauzulí

čtení ze souboru, sat solver, zápis výsledku do souboru

2.1 Generování vstupních instancí

Přestože hlavním cílem této práce není generování SAT kódování jednotlivých problémů, ale generování klauzulí pro rozbíjení symetrií nad již existujícími SAT formulemi, bylo pro experimentální část nutné vytvořit také nástroj pro generování testovacích instancí. Ten slouží především k automatizovanému vytváření vstupních formulí pro vybrané problémy popsané v předchozí kapitole.

Program je implementován jako samostatný skript spouštěný z příkazové řádky. Při spuštění musíme zadat parametry pro všechny použité problémy. Jednotlivé parametry určují velikost generovaných instancí pro všechny použité problémy. Konkrétně se jedná o počty holubů a holubníků pro problém holubníku, parametr n pro problém N královen, rozměry instancí problému nalezení matice se zadanými součty řádků a sloupců, rozměr latinského čtverce a parametry problému společenských golfistů.

Samotné generování SAT formulí přímo odpovídá kódováním uvedeným v sekci 1.3. Pro každý z problémů je implementována samostatná funkce vytvářející množinu klauzulí podle příslušných podmínek. Jelikož se jedná pouze o technickou realizaci již popsanych formulací, nebudeme zde jednotlivé algoritmy detailně rozebírat.

Úřčitou výjimku představuje problém nalezení matice s řádky a sloupci se zadanými součty. Obecně lze zadat libovolné posloupnosti požadovaných řádkových a sloupcových součtů, jejichž celkové součty jsou shodné. Pro účely experimentů však byly tyto hodnoty generovány automaticky náhodným způsobem. Nejprve jsou nezávisle vygenerovány náhodné součty pro zadaný počet řádků a sloupců. Následně je podle potřeby přidán jeden dodatečný řádek nebo sloupec tak, aby byl celkový součet řádkových a sloupcových požadavků stejný. Tím je zajištěno, že vzniklá instance splňuje základní nutnou podmínku existence řešení.

Vygenerované SAT formule jsou ukládány ve standardním formátu DIMACS CNF, který představuje standard pro výměnu SAT instancí mezi jednotlivými nástroji. Soubor obsahuje hlavičku určující počet proměnných a klauzulí a dále seznam jednotlivých klauzulí. Každá klauzule je zapsána jako posloupnost celých čísel reprezentujících literály a je ukončena hodnotou 0. Literály jsou celými čísly reprezentovány v celé implementaci nástroje. Kladná hodnota označuje proměnnou, záporná hodnota její negaci. Pro každý problém je vygenerován samostatný soubor tvaru „input{*problém*}.cnf“.

Formát DIMACS zároveň umožňuje používat komentáře zapsané v souboru řádkem začínajícím znakem „c“. Této možnosti je v implementaci využito k ukládání doplňujících informací, které nejsou součástí samotné SAT formule, ale jsou potřebné při následném generování klauzulí pro rozbíjení symetrií. Do komentářů jsou ukládány zejména údaje o rozměrech binární matice reprezentující

řešení problému a o typu uvažované symetrie. Dále jsou v komentářích ukládány parametry zadaného problému, které se využívají pro pojmenování výstupních souborů po generování klauzulí pro rozbíjení symetrií.

Informace předávané v komentářích jsou pro nás klíčové, jelikož předpokládáme, že známe symetrie zadaného problému, ale nástroj pro generování klauzulí pro rozbíjení symetrií ze samotné zadané formule tyto informace není schopen získat.

2.2 Generování klauzulí pro rozbíjení symetrií

Hlavní část implementace je soustředěna v souboru `sym.py`, který představuje vlastní nástroj pro generování klauzulí pro rozbíjení symetrií. Na rozdíl od programu popsaného v předchozí sekci již tento nástroj nepracuje s konkrétním problémem, ale pouze s jeho SAT formulací a informacemi o příslušných symetriích. Díky tomu jej lze použít pro libovolný problém, pro který známe odpovídající množinu symetrií a vhodnou reprezentaci proměnných binární maticí.

Stejně jako program pro generování vstupních instancí je i tento nástroj spouštěn z příkazové řádky. Při spuštění je potřeba zadat vstupní soubor ve formátu DIMACS, zvolenou množinu permutací, maximální hloubku generování klauzulí k_{max} a informaci o tom, zda bude výsledná formule použita pouze pro rozhodování splnitelnosti, nebo také pro počítání všech řešení.

Po načtení vstupního souboru program získá z komentářů informace uložené při generování instance. Jedná se zejména o rozměry binární matice reprezentující řešení problému a o typ symetrie, se kterým má dále pracovat. Tyto údaje jsou následně využity při vytvoření matice proměnných a při volbě odpovídajícího způsobu generování permutací. Dále jsou načteny parametry původního problému, které se využívají jen při vytváření názvů výstupních souborů.

Samotné generování klauzulí pro rozbíjení symetrií je realizováno funkcí `SYMMETRYCLAUSES`, která je uvedena v Algoritmu 4. Této funkci je předána matice proměnných vygenerována funkcí `GETMATRIX` (Algoritmus 1), typ symetrie, zvolená množina permutací a parametr k_{max} . Funkce si udržuje seznam již vygenerovaných klauzulí a zároveň eviduje pomocné proměnné vytvářené během převodu vzorů do CNF.

Další postup závisí na typu zadané symetrie. Podle toho, zda pracujeme se symetriemi na řádcích, sloupcích, obou dimenzích současně nebo se speciálními typy symetrií používanými u některých problémů, je postupně procházena odpovídající množina zpermutovaných matic. Pro každou takto získanou permutaci je následně původní matice porovnána s její zpermutovanou variantou, z čehož získáváme klauzule pomocí vzorů způsobem popsaným v první kapitole.

Vlastní generování klauzulí probíhá opakovaným voláním funkce `GETCLAUSES`, která je uvedena v Algoritmu 3. Tato funkce postupně vytváří vzory odpovídající jednotlivým možnostem, jak může být zpermutované řešení lexikograficky menší než původní řešení. Z těchto vzorů jsou následně odvozeny CNF klauzule reprezentující Lex-Leader podmínky. Všechny takto získané klauzule jsou průběžně přidávány do výsledné formule.

Funkce `SYMMETRYCLAUSES` po průchodu celým cyklem už jen vrátí množinu vygenerovaných klauzulí do funkce `MAIN`, kde se společně s celým obsahem vstupního souboru a komentářem obsahující dobu, kterou trvalo generování klauzulí, uloží do výstupního souboru opět ve formátu DIMACS. Název tohoto souboru je

tvary "output{problém}_{parametry}_{typ permutace}_{k_{max}}.cnf" pro odlišení jednotlivých běhů s různými parametry.

Popsaný postup představuje pouze obecný přehled fungování implementace. Jednotlivé části algoritmu, způsob generování permutací i samotný převod vzorů na SAT klauzule budou podrobně rozebrány v následujících sekcích.

2.3 Pomocné funkce

Než přejdeme k samotnému generování klauzulí pro rozbíjení symetrií, popíšeme nejprve několik pomocných funkcí, které jsou využívány v hlavní části implementace. Jedná se především o funkce pro generování zvolených množin permutací a dále o funkce pro vytvoření matice proměnných a aplikaci permutací na tuto matici.

První skupinu tvoří funkce generující množiny permutací, ze kterých se následně vytvářejí Lex-Leader podmínky. Funkce ALLPERM generuje všechny permutace množiny $\{0, \dots, n-1\}$. Tato volba odpovídá úplnému rozbití symetrie vzhledem k danému typu symetrie, kterou jsme dostali v zadání. Pro tuto množinu permutací ale můžeme potenciálně dostat řádově až $n!$ klauzulí, takže je použitelná jen pro velmi malá zadání.

Funkce TRANSPOSITIONS generuje všechny transpozice, tedy všechny permutace, které prohodí právě dvě pozice a ostatní ponechají beze změny. Oproti všem permutacím jde o výrazně menší množinu, jelikož není těžké nahlédnout, že počet všech transpozic na n prvcích je $\frac{n(n-1)}{2}$.

Funkce NEIGHBORS generuje pouze transpozice sousedních pozic. Pro n prvků tedy vznikne množina $\{(0,1), (1,2), \dots, (n-2, n-1)\}$. Tyto sousední transpozice tvoří standardní množinu generátorů symetrické grupy (S_n). Právě proto je tato množina důležitá při porovnávání vlivu generujících množin permutací na kvalitu částečného rozbití symetrie.

Funkce RANDTRANS generuje náhodně zvolenou množinu $n-1$ transpozic. Tato množina má stejnou velikost jako množina sousedních transpozic, a proto ji lze využít jako přirozené porovnání s funkcí NEIGHBORS. Rozdíl je v tom, že transpozice nejsou voleny podle pevné struktury sousedních prvků, ale náhodně.

Funkce NONGEN generuje množinu náhodných transpozic, která negeneruje celou symetrickou grupu (S_n). Transpozic je opět vždy celkem $n-1$, takže při porovnání s NEIGHBORS a RANDTRANS lze sledovat, zda je pro kvalitu částečného rozbití důležitá vlastnost, že množina permutací generuje danou grupu. Že množina negeneruje symetrickou grupu je zajištěno jednoduše tím, že se při generování jedna z permutovaných hodnot vyřadí a nemůže se objevit v žádné z transpozic.

Funkce GEN2 vrací dvouprvkovou množinu generátorů symetrické grupy. Ta se skládá z cyklu délky n a jedné transpozice. Tato volba je zajímavá především tím, že generuje celou symetrickou grupu pomocí velmi malého počtu permutací.

Funkce DIHEDRALGEN je v této práci určena pro problém N královen. V tomto případě nelze libovolně permutovat řádky a sloupce, protože by se obecně nezachovaly diagonální podmínky. Symetrie šachovnice jsou proto reprezentovány pomocí rotace o (90°) a reflexe podle svislé osy. Tyto dvě permutace generují dihedrální grupu symetrií čtverce. Tuto množinu permutací by samozřejmě bylo možné použít i pro jiné problémy s tímto typem symetrie.

Poslední speciální funkcí je SUMSNEIGHBORS, která je určena pro problém nalezení matice s řádky a sloupci se zadanými součty. V tomto problému nelze libovolně permutovat všechny řádky a sloupce, ale pouze ty, které mají stejný požadovaný součet. Funkce proto nejprve rozdělí indexy podle odpovídajících hodnot součtů a následně v každé takto vzniklé skupině generuje sousední transpozice.

Druhou skupinu pomocných funkcí tvoří funkce GETMATRIX a GETPERM. Funkce GETMATRIX, uvedená v Algoritmu 1, vytvoří $(m \times n)$ matici proměnných. Proměnné jsou do matice zapisovány postupně po řádcích, tedy stejným způsobem, jakým jsme v teoretické části definovali operaci (vec). Díky tomu je pořadí proměnných v matici přímo kompatibilní s lexikografickým porovnáváním používaným při generování Lex-Leader podmínek.

Funkce GETPERM, uvedená v Algoritmu 2, aplikuje na zadanou matici permutaci řádků a permutaci sloupců. Pro každý prvek původní matice určí jeho novou pozici podle zadaných permutací a vytvoří tak zpermutovanou matici. Tato funkce se následně používá při vytváření dvojice matic, jejichž vektorizace se porovnávají ve funkci GETCLAUSES.

Algoritmus 1 Vytvoření matice proměnných

```

1: function GETMATRIX( $m, n$ )
2:    $variables \leftarrow m \times n$  nulová matice
3:    $idx \leftarrow 1$ 
4:   for  $i \leftarrow 0$  to  $m - 1$  do
5:     for  $j \leftarrow 0$  to  $n - 1$  do
6:        $variables_{i,j} \leftarrow idx$ 
7:        $idx \leftarrow idx + 1$ 
8:     end for
9:   end for
10:  return  $variables$ 
11: end function

```

Algoritmus 2 Aplikace permutací na matici

```

1: function GETPERM( $matrix, \pi_r, \pi_c$ )
2:    $m \leftarrow \# \text{řádků } matrix$ 
3:    $n \leftarrow \# \text{sloupců } matrix$ 
4:    $varperm \leftarrow m \times n$  nulová matice
5:   for  $i \leftarrow 0$  to  $m - 1$  do
6:     for  $j \leftarrow 0$  to  $n - 1$  do
7:        $varperm_{\pi_r(i), \pi_c(j)} \leftarrow matrix_{i,j}$ 
8:     end for
9:   end for
10:  return  $varperm$ 
11: end function

```

2.4 Generování klauzulí z porovnání dvou vektorů proměnných

Nejdůležitější částí implementace je funkce `GETCLAUSES`, uvedená v Algoritmu 3. Tato funkce dostane na vstupu dva vektory proměnných a a b , kde a odpovídá vektorizaci původní matice a b vektorizaci její zpermutované varianty. Cílem je vygenerovat klauzule odpovídající podmínce $a \leq_{\text{lex}} b$.

Jak již bylo popsáno v teoretické části, místo přímého kódování této podmínky postupně konstruujeme opačnou situaci, tedy možnost, že b je lexikograficky menší než a . Jednotlivé kroky odpovídají tomu, že b je menší než a na i -té pozici pro krok i . Pro klauzule pak použijeme negace těchto podmínek.

Funkce postupně prochází pozice $i = 0, \dots, \min(k_{\max} - 1, n - 1)$. Parametr $k_{\max}$ zde určuje maximální hloubku, do které se vzory generují. Pokud je tedy $k_{\max}$ menší než délka vektoru, neuvažují se všechny možné pozice, ale pouze určitá počáteční část vektorů. Tím lze regulovat počet vygenerovaných klauzulí.

V každém kroku se uvažuje vzor odpovídající situaci, kdy jsou vektory a a b na všech předchozích pozicích stejné a na pozici i platí $b_i < a_i$. Jelikož pracujeme s binárními proměnnými, znamená tato ostrá nerovnost podmínky $b_i = 0$ a $a_i = 1$. Pokud je proměnná a_i rovna proměnné b_i , daná pozice nemůže být první pozicí, na které je b ostře menší než a , a proto se přeskočí.

Při konstrukci vzoru se dále udržuje množina aktivních proměnných. Aktivní proměnné jsou právě ty proměnné, které se v daném kroku skutečně podílejí na podmínkách určujících vzor. Jedná se tedy o proměnné vystupující v a a b na pozicích před i , kde požadujeme rovnost odpovídajících složek vektorů, a o proměnné a_i a b_i , které určují první ostrou nerovnost.

Rovnosti mezi proměnnými, které vznikají z podmínek $a_j = b_j$ pro $j < i$, a dále vztahy $a_i = 1, b_i = 0$, jsou ukládány pomocí datové struktury `Disjoint-Set`, známé také jako `Union-Find`. V implementaci je k tomu využita externí knihovna `DisjointSet`. Tato struktura umožňuje efektivně reprezentovat třídy ekvivalencí mezi proměnnými a speciálními reprezentanty hodnot 0 a 1. Díky tomu lze snadno zjistit, které proměnné musí mít v daném vzoru stejnou hodnotu, případně zda některá proměnná musí být rovna konstantě.

Po vytvoření všech rovností pro daný vzor se ověří, zda nevznikl spor mezi hodnotami 0 a 1. Pokud by se v jedné třídě ekvivalence ocitly současně reprezentanty obou těchto hodnot, znamenalo by to, že daný vzor není splnitelný. V takovém případě není potřeba pro tento vzor generovat žádnou klauzuli a algoritmus pokračuje dalším krokem.

Pro všechny aktivní proměnné se následně určí kanonický reprezentant jejich třídy ekvivalence. Tyto reprezentanty jsou v implementaci uloženy v množině *canon* a jsou určeny pomocí funkce z knihovny `DisjointSet`. Kanonické prvky slouží k tomu, aby bylo možné efektivně rozlišit, zda je daná proměnná ekvivalentní hodnotě 0, hodnotě 1, nebo jiné proměnné.

Pokud je proměnná v daném vzoru ekvivalentní hodnotě 0, přidá se do výsledné klauzule její kladný literál. Analogicky, pokud je proměnná ekvivalentní hodnotě 1, přidá se do klauzule její záporný literál. Výsledná klauzule tedy představuje negaci celého vzoru.

Zbývajícím případem je situace, kdy je proměnná ekvivalentní jiné proměnné, ale nikoliv přímo konstantě. V takovém případě je potřeba v negaci vzoru vyjádřit

podmínku, že tyto dvě proměnné nemají stejnou hodnotu. Pro tuto nerovnost se zavádí pomocná proměnná reprezentující vztah $(x \neq y)$ pro proměnné x a y . Aby nebylo nutné pro stejnou dvojici proměnných vytvářet pomocnou proměnnou opakovaně, uchovává se jejich reprezentace ve struktuře *aux*.

Převod těchto pomocných podmínek do CNF tvaru je proveden pomocí Tseitinovy transformace. Pro novou pomocnou proměnnou t , která reprezentuje vztah $x \neq y$, jsou přidány klauzule zajišťující ekvivalenci $t \leftrightarrow (x \neq y)$. Díky tomu lze v klauzuli použít pouze literál t , zatímco samotná definice vztahu t a $x \neq y$ je zachycena samostatnými klauzulemi.

Po zpracování všech aktivních proměnných se výsledná klauzule přidá do množiny všech vygenerovaných klauzulí pro tuto dvojici vektorů proměnných. Tato klauzule zakazuje $b <_{\text{lex}}^i a$. Opakováním tohoto postupu pro všechny uvažované pozice i získáme klauzule, které zakazují, aby zpermutovaná varianta byla lexikograficky menší než původní řešení. Tím je dosaženo požadované podmínky $a \leq_{\text{lex}} b$, případně její omezené varianty určené parametrem k_{max} .

Algoritmus 3 Generování klauzulí z porovnání dvou vektorů proměnných

```
1: function GETCLAUSES( $a, b, k_{\max}, aux$ )
2:    $C \leftarrow \emptyset$ 
3:   for  $i = 0, \dots, \min(k_{\max} - 1, n - 1)$  do
4:     if  $a_i = b_i$  then
5:       continue
6:     end if
7:      $c \leftarrow c \cup \{(\neg a_i \vee b_i)\}$ 
8:     Zkonstruujeme všechny rovnosti a nerovnosti odpovídající vzoru  $pat_i$ 
9:     Přidáme podmínky  $a_i = 1$  a  $b_i = 0$ 
10:    Určíme třídy ekvivalencí proměnných
11:    if  $0 \equiv 1$  then
12:      continue
13:    end if
14:    for all aktivní proměnné  $x$  do
15:      if  $x \equiv 0$  then
16:         $c \leftarrow c \cup \{x\}$ 
17:      else if  $x \equiv 1$  then
18:         $c \leftarrow c \cup \{\neg x\}$ 
19:      else if  $x \equiv y$  pro  $x \neq y$  then
20:        if  $\exists t \in aux$  takové, že  $t$  reprezentuje  $x \neq y$  then
21:           $c \leftarrow c \cup \{t\}$ 
22:        else
23:           $t :=$  pomocná proměnná reprezentující  $x \neq y$ 
24:           $C \leftarrow C \cup \{\text{klauzule definující reprezentaci pro } t\}$ 
25:           $c \leftarrow c \cup \{t\}$ 
26:        end if
27:      end if
28:    end for
29:     $C \leftarrow C \cup \{c\}$ 
30:  end for
31:  return ( $C, aux$ )
32: end function
```

2.5 Generování klauzulí pro rozbíjení symetrií

Tato funkce na vstupu dostává matici proměnných, typ symetrie, množinu permutací $perms$ a parametr $k_{\max}$. Množina $perms$ je vytvořena vybranou funkcí z funkcí popsanych v sekci 2.3. Podle typu symetrie funkce rozhodne, jakým způsobem budou tyto permutace aplikovány na binární matici reprezentující zadaný problém. Dále je v této funkci definována a udržována struktura pomocných proměnných aux .

Pro každou uvažovanou permutaci je pomocí funkce GETPERM vytvořena zpermutovaná varianta matice. Původní i zpermutovaná matice jsou následně převedeny na vektory a předány funkci GETCLAUSES, která vygeneruje odpovídající klauzule. Ty jsou průběžně ukládány do výsledné množiny klauzulí.

Funkce SYMMETRYCLAUSES tak tvoří zastřešující vrstvu celé implementace.

Zatímco samotná konstrukce klauzulí probíhá ve funkci `GETCLAUSES`, úkolem Algoritmu 4 je zajistit správné generování a zpracování všech permutací odpovídajících zadané symetrii.

Algoritmus 4 Generování klauzulí pro rozbíjení symetrií

```

1: function SYMMETRYCLAUSES(matrix, symtype, perms,  $k_{max}$ )
2:   if  $k_{max} = 0$  then
3:     return  $\emptyset$ 
4:   end if
5:    $C \leftarrow \emptyset$ 
6:    $aux \leftarrow$  struktura pomocných proměnných
7:   for all  $(\pi_r, \pi_c) \in perms$  do
8:      $M' \leftarrow \text{GETPERM}(M, \pi_r, \pi_c)$ 
9:      $a \leftarrow \text{vec}(M)$ 
10:     $b \leftarrow \text{vec}(M')$ 
11:     $(C_\pi, aux) \leftarrow \text{GETCLAUSES}(a, b, k_{max}, aux)$ 
12:     $C \leftarrow C \cup C_\pi$ 
13:  end for
14:  return  $C$ 
15: end function

```

3 Experimentální část

porovnání jednotlivých generátorů, generátorů a negenerátorů, jednotlivé problémy

porovnání počtu řešení před a po symmetry breakingu

Závěr

Literatura

1. ANDERS, Markus; BRENNER, Sofia; RATTAN, Gaurav. satsuma: Structure-based Symmetry Breaking in SAT. *CoRR*. 2024, roč. abs/2406.13557. Dostupné z DOI: 10.48550/ARXIV.2406.13557.
2. DEVRIENDT, Jo; BOGAERTS, Bart; BRUYNOOGHE, Maurice; DENECKER, Marc. Improved Static Symmetry Breaking for SAT. In: CREIGNOU, Nadia; LE BERRE, Daniel (ed.). *Theory and Applications of Satisfiability Testing – SAT 2016*. Cham: Springer International Publishing, 2016, s. 104–122. ISBN 978-3-319-40970-2.
3. FLENER, Pierre; FRISCH, Alan; HNICH, Brahim; KIZILTAN, Zeynep; MIGUEL, Ian; PEARSON, Justin; WALSH, Toby. Breaking Row and Column Symmetries in Matrix Models. In: 2002. ISBN 978-3-540-44120-5. Dostupné z DOI: 10.1007/3-540-46135-3_31.
4. GENT, Ian P.; PETRIE, Karen E.; PUGET, Jean-François. Symmetry in Constraint Programming. In: ROSSI, Francesca; BEEK, Peter van; WALSH, Toby (ed.). *Handbook of Constraint Programming*. Elsevier, 2006, sv. 2, s. 329–376. Foundations of Artificial Intelligence. Dostupné z DOI: 10.1016/S1574-6526(06)80014-3.
5. CODISH, Michael; JANOTA, Mikoláš. *Breaking Symmetries from a Set-Covering Perspective*. 2025. Dostupné z arXiv: 2502.10056 [cs.LO].
6. SILVA, João; LYNCE, Inês; MALIK, Sharad. Conflict-Driven Clause Learning SAT Solvers. *Frontiers in Artificial Intelligence and Applications*. 2009, roč. 185. Dostupné z DOI: 10.3233/978-1-58603-929-5-131.
7. DEVRIENDT, Jo; BOGAERTS, Bart. *BreakID: Static Symmetry Breaking for ASP (System Description)*. 2016. Dostupné z arXiv: 1608.08447 [cs.AI].
8. HAKEN, Armin. The Intractability of Resolution. *Theor. Comput. Sci.* 1985, roč. 39, s. 297–308.
9. ROSSI, Francesca. *Handbook of constraint programming*. Ed. PETER, Van Beek; ROSSI, Francesca; VAN BEEK, Peter; WALSH, Toby. Elsevier Science & Technology, 2014. Foundations of Artificial Intelligence.
10. LAGNIEZ, Jean-Marie; MARQUIS, Pierre. An Improved Decision-DNNF Compiler. In: *Proceedings of the Twenty-Sixth International Joint Conference on Artificial Intelligence, IJCAI-17*. 2017, s. 667–673. Dostupné z DOI: 10.24963/ijcai.2017/93.
11. BIERE, Armin; FALLER, Tobias; FAZEKAS, Katalin; FLEURY, Mathias; FROLEYKS, Nils; POLLITT, Florian. CaDiCaL, Gimsatul, IsaSAT and Kissat Entering the SAT Competition 2024. In: HEULE, Marijn; ISER, Markus; JÄRVISALO, Matti; SUDA, Martin (ed.). *Proc. of SAT Competition 2024 – Solver, Benchmark and Proof Checker Descriptions*. University of Helsinki, 2024, sv. B-2024-1, s. 8–10. Department of Computer Science Report Series B. Dostupné také z: <http://hdl.handle.net/10138/584822>.
12. TSEITIN, G. S. On the complexity of derivations in the propositional calculus. *Studies in Constructive Mathematics and Mathematical Logic*. 1968.

Seznam obrázků

Seznam tabulek

Seznam použitých zkratek

A Přílohy

A.1 První příloha